

CS633 Assignment 1

Group 46

Sruthi (230751)

Sharmila (230620)

Meghana(230512)

Hemanth Kumar(230128)

Karthik(230568)

February 15, 2026

1 Code Description

1.1 Overview

The program implements an **MPI-based parallel data exchange and computation scheme** among **P processes** using only **point-to-point communication**. Each process can act as a **sender** to ranks at distances **D1** and **D2** and as a **receiver** for ranks at distances $-D1$ and $-D2$. For **T iterations**, data is exchanged, processed at receiver ranks, returned to senders, and updated for the next iteration. Finally, all valid senders report their **maximum received values** to **rank 0**, which prints the global maxima and total execution time.

1.2 Initialization

MPI is initialized and rank information is obtained using:

```
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &P);
```

The program reads five command-line inputs:

```
int M = atoi(argv[1]);
int D1 = atoi(argv[2]);
int D2 = atoi(argv[3]);
int T = atoi(argv[4]);
int seed = atoi(argv[5]);
```

Random initial data is generated using:

$$(double) \text{rand}() \times (\text{rank} + 1) / 10000.0.$$

1.3 Iterative Communication

The computation runs for **T iterations**. A process sends data only if the destination rank exists:

```
valid_D1 = (rank + D1 < P);
valid_D2 = (rank + D2 < P);
```

An even-odd ordering prevents deadlock and uses only:

```
MPI_Send(...);  
MPI_Recv(...);
```

1.4 Receiver Computation

Receivers process the received **M doubles** as follows:

D1 receiver (squaring):

```
for (int i = 0; i < M; i++)  
    temp[i] = temp[i] * temp[i];
```

D2 receiver (logarithm):

```
for (int i = 0; i < M; i++)  
    temp[i] = log(temp[i]);
```

The processed data is sent back to the sender using `MPI_Send`.

1.5 Sender Update

After receiving processed values, buffers are updated for the next iteration:

D1 update:

```
buffer_D1[i] = (unsigned long long)(recv_D1[i]) % 100000;
```

D2 update:

```
buffer_D2[i] = recv_D2[i] * 100000;
```

Thus, each iteration follows:

communication → computation → return → update.

1.6 Final Maximum Collection

Each valid sender computes the maximum of final received values:

```
if (recv_D1[i] > maxD1) maxD1 = recv_D1[i];  
if (recv_D2[i] > maxD2) maxD2 = recv_D2[i];
```

These maxima are sent to rank 0 using:

```
MPI_Send(&maxD1, 1, MPI_DOUBLE, 0, 10, MPI_COMM_WORLD);  
MPI_Send(&maxD2, 1, MPI_DOUBLE, 0, 11, MPI_COMM_WORLD);
```

Rank 0 computes the global maximum from all valid senders.

1.7 Timing and Output

Execution time is measured using:

```
double start = MPI_Wtime();  
double end   = MPI_Wtime();  
MPI_Reduce(&localTime, &totalTime, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
```

Finally, rank 0 prints:

```
printf("%lf %lf %lf\n", globalMaxD1, globalMaxD2, totalTime);
```

2 Code Compilation and Execution Instructions

The program is written in C using the MPI library and was executed on a Linux environment with OpenMPI installed.

2.1 Compilation

Navigate to the source directory and compile using the MPI compiler wrapper:

```
mpicc src.c -o src -lm
```

2.2 Execution

The program is executed using `mpirun` with the required number of processes P and data size M :

```
mpirun -np <P> ./src <M> <D1> <D2> <T> <Seed>
```

2.3 Configurations Used for Plot

The following configurations were used for performance evaluation:

$$P \in \{8, 16, 32\}, \quad M \in \{262144, 1048576\}$$

Each configuration was executed five times. The program outputs (D1, D2, and execution time) were collected using SLURM job scripts.

2.4 Running Job Scripts

The experiments were performed on the cluster using the following commands:

```
sbatch job_P8_M262144.sh
sbatch job_P8_M1048576.sh
sbatch job_P16_M262144.sh
sbatch job_P16_M1048576.sh
sbatch job_P32_M262144.sh
sbatch job_P32_M1048576.sh
```

Each job script stores the execution output into corresponding `results_P#_M#.txt` files.

2.5 Collecting Timing Data

After all jobs completed, the results were combined into a single timing dataset using the provided shell script:

```
./timing.sh
```

This script concatenates all result files and generates the file `timing_data.txt`, which is used for further analysis.

2.6 Generating the Plot

The boxplot of execution times was generated using a Python script:

```
python3 plot.py
```

The script reads the timing dataset and produces the execution time distribution plot included in the Results section.

3 Results

The distribution of execution times over five runs for each configuration is shown in Figure 1. The experiment was performed for two data sizes, $M = 262,144$ and $M = 1,048,576$, using $P = 8, 16$, and 32 MPI processes. Execution time increases with both process count and message size.

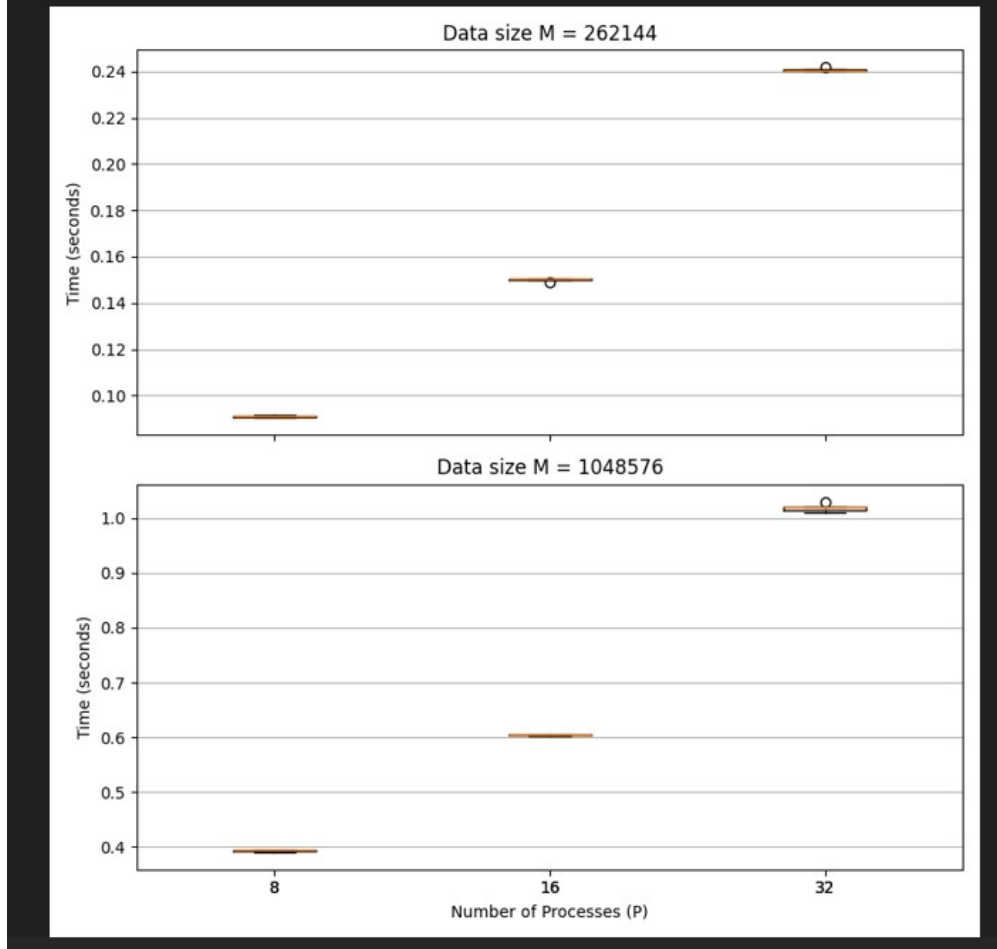


Figure 1: Execution time distribution (5 runs per configuration). The Y-axis shows execution time (seconds) and the X-axis shows process count P for two data sizes M .

Processes (P)	Data Size (M)	Avg. Time (s)	Max D1 / Max D2
8	262,144	0.0909	9.987204×10^9 / 14.1636
8	1,048,576	0.3935	9.987204×10^9 / 14.1636
16	262,144	0.1500	9.987204×10^9 / 14.1636
16	1,048,576	0.6041	9.987204×10^9 / 14.1636
32	262,144	0.2408	9.987204×10^9 / 14.1636
32	1,048,576	1.0159	9.987204×10^9 / 14.1636

Table 1: Timing summary averaged over five runs per configuration.

3.1 Analysis of Results

Correctness Verification: The maximum values D1 and D2 are identical for all process counts ($P = 8, 16, 32$) and both data sizes. This confirms that the parallel implementation is functionally correct and independent of the number of processes. The MPI communication preserves data consistency across all executions.

Effect of Process Count: For both datasets, execution time increases as the number of processes increases. Ideally, parallelization should reduce runtime, but the opposite behavior is observed. This indicates that communication overhead dominates computation in the implemented point-to-point MPI workflow.

Communication Overhead: As P increases, each process performs less computation but exchanges more messages. Each communication introduces latency and synchronization cost. For this relatively small computational workload, the communication cost exceeds computational savings, leading to increased total runtime.

Scaling with Data Size: Increasing the data size from 262,144 to 1,048,576 increases execution time by approximately four times across all process counts. This shows runtime is proportional to message volume, which is expected for a communication-bound MPI application.

Inter-node Latency: The highest execution time occurs at $P = 32$. At this configuration, processes are distributed across multiple compute nodes rather than a single node. Communication therefore occurs over the network instead of shared memory, introducing network latency and bandwidth limitations. Hence, inter-node communication significantly degrades performance compared to intra-node communication.

Overall Observation: The experiment demonstrates poor strong scaling. The application is not computation-intensive enough to benefit from additional processors, and increasing the number of processes mainly increases communication and synchronization overhead rather than improving performance.

4 Conclusions

This experiment evaluated the performance of a point-to-point MPI parallel program using different process counts ($P = 8, 16, 32$) and two message sizes ($M = 262,144$ and $M = 1,048,576$).

The identical D1 and D2 values across all executions confirm that the parallel implementation is correct and independent of the number of processes. Hence, the MPI communication and synchronization were implemented properly.

However, performance did not improve with increasing process count. Instead, execution time increased as more processes were used. This indicates that the application is communication-bound rather than computation-bound. The overhead of message passing, synchronization, and network latency dominates the computation time.

Execution time scaled approximately linearly with data size, showing that runtime is mainly determined by communication volume. Furthermore, the worst performance occurred at $P = 32$, where processes were distributed across multiple nodes. Inter-node communication introduced significant network latency compared to intra-node shared memory communication.

Overall, the experiment demonstrates poor strong scaling. Increasing the number of processors does not always improve performance, especially for workloads with low computation and high communication cost. Effective parallelization therefore requires a sufficiently large computational workload to amortize communication overhead.

All members of the group contributed equally to this assignment. Each member participated in the implementation, execution of experiments, analysis of results, and preparation of the report.